



**Department of Electrical,
Computer, & Biomedical Engineering**
Faculty of Engineering & Architectural Science

COE/ELE 70AB Milestones Compliance Report (MCR)

Project Title	Heterogeneous CPU-GPU Implementation of Collision Detection for Gaming Optimizing Collision Detection with the GPU for Lagrangian (Particle Based) Fluid Simulations for Real-Time Gaming Applications in Unity
MCR Number	1
Project Manager for the MCR period	Student C: Cameron Tuffner-Lyons
Team Players for the MCR period	Names of the rest of the team other than the project manager: Student A: Victor Do Student B: Jj Marr Student D: Davis Cheung
Faculty Supervisor	Your FLC Name: Gul Khan

- Tasks Outlined for the Reporting Period (e.g. MCR I – Weeks 4 & 5)** (Provide detailed information on the tasks to be completed for the reporting period as per the milestone submitted to your FLC in Week3)

Group: GK02

Student A:

1. Project Scoping

Decide on a suitable topic for demonstrating Heterogeneous CPU-GPU Implementation of Collision Detection for Gaming:

Completed a thorough evaluation of different gaming scenarios where heterogeneous computing could provide performance gains, particularly focusing on collision detection within fluid simulations.

Finalize the topic:

After reviewing potential case studies, finalized on implementing a particle-based fluid simulation with GPU-accelerated collision detection in a real-time game environment.

Research fluid simulations and collision detection:

Studied academic papers and online resources on fluid simulations, focusing on the differences between particle-based and grid-based methods.

Justify the topic's relevance:

Documented how GPU-accelerated particle-based simulations are ideal for showcasing the parallel processing power of modern GPUs, aligning with current gaming industry trends.

Establish project goals and scope:

Defined clear objectives for the project, including demonstrating performance improvements in real-time simulations through GPU acceleration. Key metrics such as frame rate and CPU-GPU workload balance were established.

Student B:

Unity vs Unreal Engine Comparison:

Research Cost Considerations:

- Compare the licensing costs and revenue models of Unity vs Unreal for small-scale developers.
- Examine potential long-term costs, such as asset store purchases, plugins, and third-party integrations.

Performance Overhead:

- Analyze the baseline performance overhead of both engines, focusing on their default rendering pipelines and post-processing effects.
- Investigate how this overhead impacts fluid simulation, especially with real-time GPU-accelerated computations.

Profiling Tools:

- Compare the profiling tools provided by Unity (e.g., Unity Profiler, Frame Debugger) vs Unreal (e.g., Unreal Insights).
- Investigate how these tools handle GPU/CPU performance bottlenecks, particularly for particle-based simulations.

Learning Curve:

- Review available learning resources for both engines (tutorials, documentation, community support).
- Interview team members to gauge familiarity and experience with each engine.

- Analyze the development time required to implement key components like fluid simulation and collision detection.

Documentation Availability:

- Assess the quality and comprehensiveness of official documentation for fluid simulations and GPU computations in both engines.
- Look into third-party resources and forums for additional support.

Platform Support:

- Review the list of supported platforms for both engines, particularly focusing on gaming platforms (PC, consoles, mobile, WebGL).
- Evaluate which engine offers better cross-platform support for fluid simulations.

Relevance to Capstone Project:

- Examine how the unique features of each engine align with the project's needs (real-time GPU-accelerated fluid simulation).
- Analyze the importance of factors like scalability, ease of prototyping, and integration with particle-based systems.

Student C:**1. Managerial Duties**

- a. setup meeting with FLC
- b. setup internal meeting with group members
- c. setup meeting for discussion regarding possible implementations

2. Graphics API evaluation: CUDA vs HLSL vs OpenCL

- a. Evaluate Hardware and Platform Compatibility
 - i. Research CUDA's compatibility with NVIDIA GPUs and its limitations.
 - ii. Explore HLSL's integration with Unity and its cross-platform translation capabilities.
 - iii. Investigate OpenCL's broader hardware support (NVIDIA, AMD, Intel) and its applicability to general-purpose computing.
- b. Ease of Use and Integration
 - i. Compare the ease of development and debugging for each option within Unity.
- c. Performance Considerations
 - i. Compare the raw performance of CUDA, HLSL, and OpenCL for compute-intensive tasks like particle-based fluid simulation.
 - ii. Analyze the ease of parallelization and performance optimizations available in each language.
- d. Cross-Platform Development

- e. Cost and Accessibility
 - i. Consider the cost implications of using CUDA, HLSL, and OpenCL, including hardware and software investments.

3. Game Engine vs No Game Engine Re-evaluation

- a. Evaluate the Benefits of Using a Game Engine.
 - i. Investigate the potential time savings and performance benefits from using pre-built physics engines, render engines, and profiling tools.
- b. Integration and User Interaction
 - i. Explore how Unity's drag-and-drop tools can simplify the creation of interactive fluid simulations, control panels, and real-time user input.

Student D:

1. Lagrangian vs Eulerian Fluid Simulations

- a. Research Parallelization Opportunities:
 - i. Investigate how each method (Lagrangian vs Eulerian) can be optimized for parallel processing on the GPU.
 - ii. Analyze the strengths and weaknesses of each method in terms of real-time performance.
- b. Evaluate Numerical Stability:
 - i. Research methods for stabilizing Lagrangian particle simulations, especially for large time steps.
 - ii. Investigate how Eulerian methods manage stability across large grid structures.
- c. Collision Detection Capabilities:
 - i. Compare how each method handles collision detection, focusing on the gaming context (e.g., interactions with objects and characters).
- d. Best Use Cases:
 - i. Analyze which simulation method is more appropriate for real-time gaming applications, considering the project's goals and platform targets.

2. Create a Theory Document to Combine All Research and Findings:

- a. Collect all references, research papers, and technical articles relevant to CPU-GPU heterogeneous computing, particle-based fluid simulations, and collision detection.
- b. Include notes on comparison points like Unity vs Unreal, CUDA/HLSL/OpenCL, and Lagrangian vs Eulerian simulations.

3. Tooling and Integration:

- a. Compare the development environments and tooling for each language (e.g., Visual Studio Code for C#, Visual Studio for C++).
 - i. Dependency on game engine evaluation, graphics API evaluation

2. **Progress Made in Reporting Period (e.g. MCR I – Weeks 4 & 5)** (Provide detailed information on the progress that you (as a group and individual) made during the reporting period. You can include figures, datasheets, flowcharts etc. and additional information as requested by your FLC. You should use your progress to justify compliance to the tasks outlined for the reporting period as per the milestones submitted to your FLC in Week3)

Group:

Student A:

Created a GitHub repository for the project:

Initialized the project's GitHub repository with a proper directory structure and version control workflows.

Set up repository structure and workflows:

Organized the repository to separate assets, scripts, and documentation. Integrated .gitignore to exclude Unity auto-generated files and set up a branching strategy.

Set up the Issues page on GitHub:

Created core project issues on GitHub, defining milestones and priorities. Tagged and assigned rough deadlines for each task.

Broke down the overall project plan into manageable tasks:

Structured the plan into smaller tasks like research, prototyping with Unity's physics engine, and developing GPU-based optimizations.

Created a prototype in Unity using the built-in physics engine:

Set up a Unity project with the necessary packages, implemented a prototype of a Lagrangian fluid simulation with basic particle collision detection using Unity's physics system, and recorded initial performance benchmarks.

Prepare the prototype for GPU integration:

Created a plan for migrating the current CPU-based collision detection to a GPU-accelerated system using Unity's existing tools and external libraries.

Finalize theory document for team review:

Completed a draft of the theory document, including research citations and links to relevant academic papers.

Project Scoping

Task: Decide on a suitable topic for demonstrating Heterogeneous CPU-GPU Implementation of Collision Detection for Gaming

Details:

Conducted a comprehensive evaluation of various gaming scenarios where heterogeneous computing could deliver significant performance gains. A thorough analysis indicated that collision detection in particle-based fluid simulations offers ideal parallelization opportunities for the GPU. This area was selected due to its highly computational nature and relevance to modern game development.

After reviewing potential applications and case studies, the project topic was finalized as the implementation of a particle-based fluid simulation, focusing on GPU-accelerated collision detection in a real-time gaming environment. This choice aligns with industry trends where fluid dynamics increasingly play a role in enhancing realism within games.

Background:

Heterogeneous computing involves using different types of processors (in this case, a CPU and GPU) to perform tasks. The **CPU** is optimized for general-purpose computing and handles tasks sequentially. It manages logic, inputs, AI, and the game's overall control flow. The **GPU** excels at parallel processing, making it ideal for graphics rendering and tasks that require many simultaneous calculations. A **heterogeneous implementation** means distributing the workload between the CPU and GPU to improve performance and efficiency.

The utility of a heterogeneous CPU-GPU system is that the CPU is responsible for handling sequential, logic-based tasks such as inputs, AI, and game state management, while the GPU handles graphics rendering and the raw computational power required for parallelized collision detection and particle physics.

Arguments for Using Particle-Based Fluid Simulation

- Particle based fluid simulations are very **compute intensive** because you need an efficient solver to compute the numerous collisions between many particles.
- It has a **Highly Parallelizable Nature** because each particle is modeled individually and there are only a handful of particles involved in each collision. Meaning there are many independent events to compute.
- There is a **vast amount of research** on optimizing particle-based simulations, offering plenty of room to explore existing methods and push forward with novel ideas.
- There are **various optimization techniques** to explore such as spacial subdivision which involves breaking up the simulation space into only the regions which are relevant for each particle to compute, reducing the number of unnecessary calculations
- “The global video game market size was estimated at USD 217.06 billion in 2022 and is expected to grow at a compound annual growth rate (CAGR) of 13.4% from 2023 to 2030.” [1]

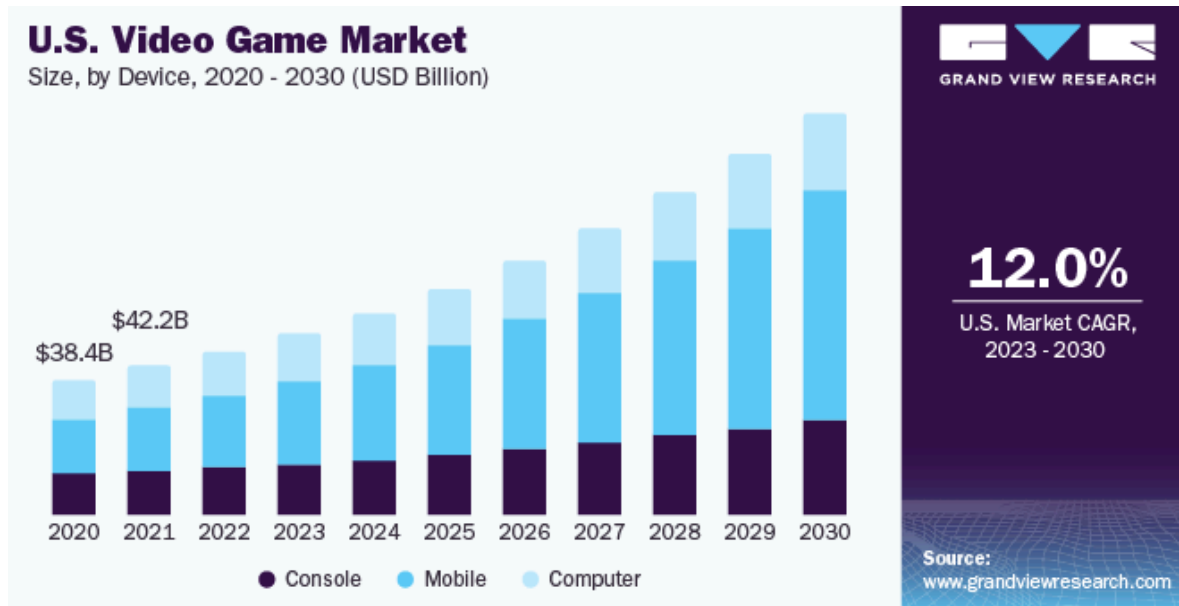


Figure 1. U.S. video game market cap projection from 2020 to 2030 [1].

- Fluid simulation, with its potential for highly realistic water and liquid interactions, plays an increasingly important role in **creating believable virtual environments**. As the industry continues to expand, developers are seeking ways to implement more sophisticated physics models that don't compromise on performance.
- "The global graphic processing unit (GPU) market size is calculated at **USD 75.77 billion in 2024** and is expected to reach around **USD 1,414.39 billion by 2034**, registering a solid CAGR of 13.8% between 2024 and 2034" [2]

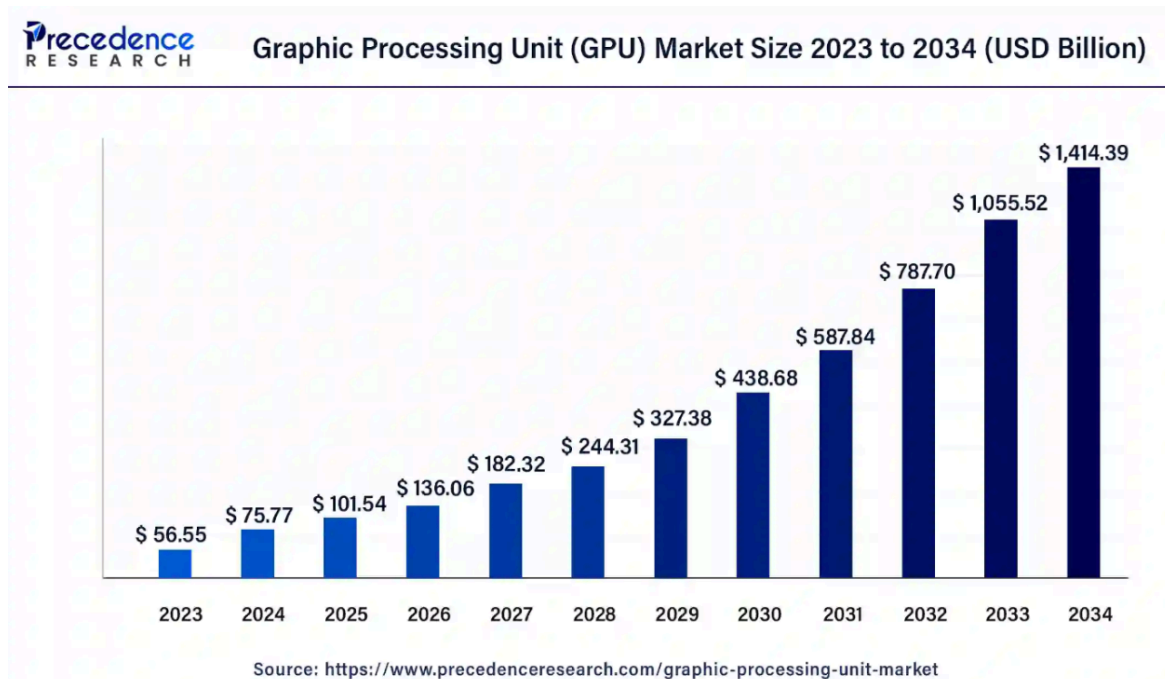


Figure 2. GPU market cap projection from 2023-2034 [2].

- As consumer-grade **GPUs become more powerful**, they are **finally becoming capable** of handling real-time fluid simulations, even in conjunction with CPU tasks. This makes the project not only relevant for current technological trends but also future-proof as hardware continues to evolve.
- As time goes on, **consumers desire** more **realistic** and **interactive** games.
- Fluid dynamics are **no longer just a visual** enhancement; they can now be a core part of **game mechanics** and environmental storytelling.

Summary:

Particle-based fluid simulations are particularly well-suited to heterogeneous computing due to their compute-intensive nature. The individual modeling of particles makes these simulations highly parallelizable, as each particle's interaction is largely independent of others. This characteristic enables efficient GPU utilization for real-time simulations. And market research indicates that this is an area that will be in demand in the future.

Does This Topic Involve Collision Detection?

Yes, the project involves collision detection. A Smoothed Particle Hydrodynamics (SPH) simulation, which we are implementing, shares many core principles with traditional collision detection systems. Techniques such as spatial subdivision and management of numerous spherical bounding boxes will be used to efficiently detect and resolve collisions between fluid particles. In addition to fluid-particle interactions, the system will handle collisions between fluids and obstacles, ensuring the inclusion of robust real-time simulation.

The CPU handles low latency and sequential work, including but not limited to:

- UI Updating
- Input Management
- Player movements
- Game logic and state management
- AI or pathfinding that rely on sequential algorithms.
- Event triggers
- Initialization/Destruction of game object entities
- Initialization of fluids in scene
- Non-parallelizable collisions such as coin collection
(Sequential tasks between one/few player(s) and one/few obj(s))
- Rigidbody physics for a small handful of obstacles.
- Other simple scripts like the ones to continuously translate/rotate an object.
- The CPU can manage the flow of data between the game engine and the GPU. It will handle synchronization between the two, such as determining when to send data to the GPU and when to retrieve results.

The GPU handles Parallelizable work:

- Millions of fluid particle to fluid particle collisions
- Fluid particle to obstacle collisions
- Other parallelizable compute tasks
 - Bitonic, Radix, Counting Sort Functions
 - Hashing Functions

Rationale for this Split:

We established that the CPU can only simulate up to 500 particles (in the worst case, our prototype) before it starts to noticeably slow down. But we want to simulate orders of magnitudes more, perhaps 200k, or 2M particles. It is not efficient to split that task up between the CPU and GPU because the GPU would be handling over 99% of that and the remaining 1% (2k) would overwhelm the CPU, preventing it from running the other necessary tasks for a game. We also want to minimize data transfer between the CPU and GPU to avoid performance bottlenecks. Only transferring necessary data, like object positions and velocities.

Thus, involving the CPU in particle interactions specifically would yield little to no performance benefit and would increase the need for data transfer between the CPU and GPU. Which is why the CPU will be focused on the low latency and sequential work and the GPU will handle Parallelizable work. This way, our final solution will be adoptable by other game developers for their projects.

Student B:

Research Cost Considerations:

Licensing costs

Both Unity and Unreal are free for individuals or organizations making small amounts of money.

Unreal

Unreal's free tier caps out at around US\$1 million in corporate annual gross revenue (AGR). Past US\$1 million in AGR, and US\$1 million in total sales for a given product, we must pay Unreal 5% of the revenue that we gain from licensing that product to end-users. Alternatively, Unreal has a US\$2500 annual licence fee per computer if we're not licensing our software to others.

As an example, if we put the fluid simulation demo in a museum kiosk, we'd pay US\$2500 a year to licence Unreal on that computer, but if we sold that fluid simulation demo to a consumer, we'd pay 5% on the sales past US\$1 million in total sales

Unity

Unity is free up to US\$200k in AGR. Past that point, Unity costs around US\$2000 per developer, and goes up to US\$5000 a developer after US\$1 million in AGR. There are no royalties.

Comparison

Unity is more cost-effective than Unreal past US\$1 million in AGR, and is less cost-effective prior to that point. Ultimately, neither option will cost us any money during the development of the capstone project.

If we were to start our own business using this technology, Unreal would likely be cheaper initially until we reach US\$1 million in AGR. However, we don't expect to commercialize the underlying technology ourselves.

Our goal is to use this product in static display environments for educational purposes, i.e. a museum or kiosk. In this scenario, paying \$2500 a year per individual kiosk would significantly increase the price point we have to sell our simulation at. This is not worth it to us.

Performance Overhead:

Unity uses C# for user additions while Unreal uses C++. A naive analysis would assume that Unity would be faster than Unreal. However, empirical evidence that we gathered when creating an empty Unity and

Unreal project showed Unity to run at 600 frames per second, while Unreal could only reach 60 fps. We also found an empty unity project builds into an executable around 1mb, while an empty unreal project builds to around 800mb because of everything that is packed into it, making file sharing and collaboration for this project more difficult.

This is because Unreal comes prepackaged with a heavy photorealistic deferred graphics renderer, a sophisticated lighting engine, and numerous post processing effects which are composited together to assist in creating realistic scenes. While Unity is packaged with a forward rendering pipeline by default which is less resource intensive. Unreal therefore includes many visual effects that we do not require in order to make an educational fluid simulation program. Either way, this means Unity is more performant than Unreal.

We also predict that an overwhelming majority of our application's processing time will be spent doing physics simulation. Based on an experiment we performed with 500 particles in a 2-dimensional box, we discovered that simulating particles takes upwards of 70% of processing time using Unity's default physics engine. Optimizing particle collision is therefore our greatest priority as we will require significantly more processing time when dealing with the hundreds of thousands of particles we need for high quality fluid simulation.

Because optimizing particle collision is our biggest priority, the C# vs C++ differences matter even less as the particle collision will be done on the GPU. This will make shader code (neither C# nor C++) the main bottleneck for our application, and so C++ would not provide a significant performance impact.

Learning Curve:

Unity has several academic papers using it to perform fluid simulations. Unreal does not. The expert judgment of our group was used to determine that Unity is also easier to write code for as it's in C# instead of C++. Unity's ecosystem is known for being beginner-friendly, with a large community, a wealth of tutorials, and comprehensive documentation, it is ideal for prototyping and rapid iterations.

C# is also much easier to port to different platforms. C++, being based on C, is platform specific in many ways. C# runs on a virtual machine like Java, which means it can work on a variety of platforms. This is our goal for this project as we want a particle simulator that can work on a variety of platforms that someone wishing to display visualizations would use.

Additionally, Unity has support for cross-compiling HLSL shaders to any target platform, such as Vulkan or OpenGL on mobile devices.

Platforms that Unity Supports [3], [4]:

- **Desktop:**
 - Windows (PC) (Also supports ARM64 Processors)
 - Mac
 - Universal Windows Platform (UWP)
 - Linux Standalone
- **Mobile:**
 - iOS
 - Android™
- **Extended Reality (XR):**
 - ARKit
 - ARCore
 - Microsoft HoloLens
 - Windows Mixed Reality
 - Magic Leap (Lumin)
 - Oculus
 - PlayStation®VR2
- **Consoles:**
 - PS5™
 - PS4™
 - Xbox®One
 - Xbox Series X|S
 - Nintendo Switch™
- **WebGL (Embedded into browser)**
- **Embedded:**
 - Embedded Linux
 - QNX

Unreal Engine already includes the Niagara Engine, which is a robust system that features Niagara Fluids, a FLIP fluid solver with more capabilities than we could realistically implement in a year. This makes our work redundant in Unreal, as developers are more likely to use these built-in, well-established tools. In contrast, Unity does not come with a built-in fluid simulation system. When searching for "Unreal Fluid Sim," you'll find a variety of tutorials on how to use the built-in fluid simulation features, making it the go-to solution for Unreal developers. However, if you search for "Unity Fluid Sim," the results primarily show custom-coded solutions that are often not well-optimized or fully featured. This creates a gap in the Unity ecosystem, as there is no single, optimal solution for fluid simulation. Our goal is to create a fully featured, well-optimized solution for Unity that other developers could easily integrate into their games, addressing this market need.

Student C:

CUDA vs HLSL vs OpenCL

Performance:

The performance of CUDA is generally higher than OpenCL but **the performance gain is only applicable to NVIDIA gpus** [5]. This performance gain was also found to be caused primarily by a difference in compiler architecture and coding methodology [6].

Overall the performance of CUDA and OpenCL can be considered equal [6].

HLSL (High Level Shader Language) is a shader language that was designed primarily for graphics applications rather than general purpose compute. HLSL implements Microsoft DirectX shaders and can incorporate both the CUDA and OpenCL libraries. As such the performance of HLSL will be comparable to both CUDA and OpenCL [7].

HLSL is best suited for graphics pipelining, especially in games as it implements the DirectX/Direct3D graphics engines and integrates directly into many game engines [6].

Cross-Platform Compatibility

CUDA is designed and optimized for NVIDIA GPUs and as such has limited cross platform compatibility, running CUDA code on non-NVIDIA GPUs requires a recompile to be able to run using AMDs HIP translation [8], [9].

Both HLSL and OpenCL run natively on NVIDIA and Radeon (and Intel) GPUs [10]. Due to HLSL using Microsoft DirectX shaders, HLSL cannot run on MacOS without a translation layer or recompile to GLSL which is handled by most game engines. All three libraries support linux operating systems with some setup.

OpenCL also has support for FPGA giving it the best cross platform compatibility over both CUDA and HLSL [11].

Cost and Accessibility

NVIDIA GPUs tend to be the most expensive consumer GPUs which also sets the cost of using CUDA roughly equal to the cost of an NVIDIA GPU [12]. NVIDIA also does not produce any integrated CPU+GPU processors while both AMD and Intel do.

Since OpenCL is supported by all major consumer GPU manufacturers, the cost of entry is very low and OpenCL code can be tested on most GPUs.

HLSL is also supported by all major consumer GPU manufacturers [10].

Both OpenCL and HLSL provide the lowest cost of entry compared to CUDA.

Ease of Use and Integration

HLSL is the most popular choice in the game development industry [13]. It has seamless integration into the Unity engine and rendering pipeline [3]. HLSL compute shaders can be written directly in Unity and can be easily translated to GLSL for cross-platform development [3]. Access to features like DirectCompute (on DirectX) increase the functionality and efficiency of the compute code when working on games [3].

OpenCL is more robust than HLSL in terms of general-purpose computing features, allowing the implementation of very advanced parallel computing techniques and can run on almost any platform [14]. Most game engines have no native support for OpenCL and require third party plugins to gain functionality [3]. Due to its lack of integration, memory management between the openCL program and the unity engine would be required, increasing complexity and processing time.

CUDA has the same limitations as OpenCL in terms of development in unity (or other game development IDEs) requiring third party plugins to gain functionality [3].

Due to its ubiquity in game development, HLSL has the simplest integration and ease of use for parallel computing and game development.

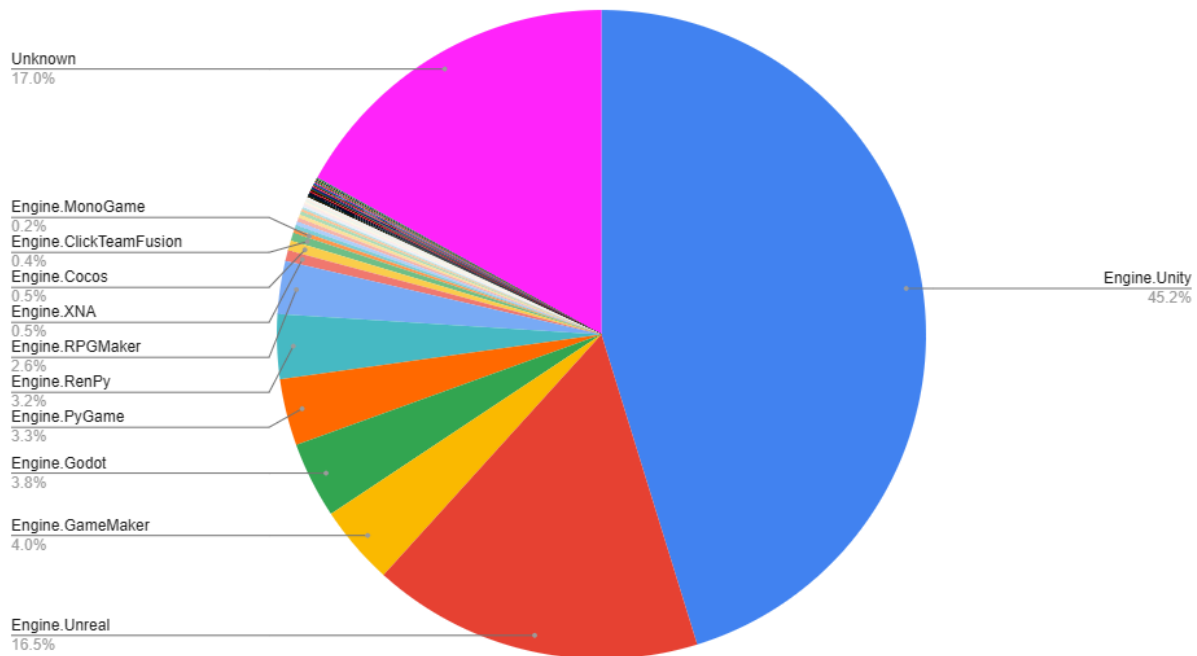
Game Engine vs No Game Engine

Game engines provide platform abstraction and allow for easy recompiling for various platforms that may be used [15]. Game engines also provide some base level functionality like input management, the ability to place shapes and objects in a 3D environment, a user interface for players and developers, as well as the ability to easily edit the scene of games, allowing the developers to put more focus into the more important aspects of game development [15]. Game engines are also optimized for real-time performance and are able to assist in optimizing the overall project for real-time execution [16]. They also give access to tools which help measure the performance of the application or project and give valuable insights that might not be available without using the engine. The component-based architecture of most game engines also makes it easy to implement new features or add third party plug-ins that can further the development of the project [15].

According to SteamDB, the game database used by the game platform Steam, the vast majority of games are created with a game engine and up to 65.2% of the games released on steam in 2024 were made using either the Unity, GameMaker or Unreal game engines [17].

All of these engines have native support for HLSL. That means by working with HLSL our solution can potentially be ported to more than 65% of games but if we use openCL our solution will only apply to our specific implementation. Also modern game engines can automatically translate HLSL to GLSL enabling support for dozens of different platforms for us [3].

Game Engine Marketshare on Steam in 2024

**Figure 3.** Game engine market share for PC gaming platform “Steam” in 2024 [17]

Using a game engine does however present some disadvantages for the programmers. If the game engine used is not open source, which most are not, then any bug in the engine will be unfixable and will require the team to wait for the bug to be patched before being able to proceed [16]. On the other hand, these engines are used by millions of developers and there are billions of dollars invested into teams which work to fix any bugs that are found. Game engines also can add computational overhead that may not be beneficial to the performance of the game, especially if the game is quite small (such as a phone game) [16]. Finally, game engines are typically free up to a generous revenue threshold [16].

For the capstone project it was decided that a game engine should be used for development as the benefits of reduced workload through pre-built modules, documentation and the cross-platform capabilities will prove to be more beneficial than the possibility of additional computational overhead.

Student D:

Lagrangian vs Eulerian Fluid Simulations

High-level Summary

Lagrangian and Eulerian fluid simulations are the two most popular methods for digitally simulating fluids. Lagrangian fluid simulations focus on modeling individual particles within the fluid, with each particle carrying its own momentum and exerting its own forces on the particles around it [18], [19], whereas Eulerian fluid simulations involve using a grid-based simulation mesh that remains stationary and models fluid properties at each point within the grid using vectors [18], [20]. An important thing to note is that Eulerian simulations do not model individual particles and divide the fluid into a discretized grid, meaning that a *pure* Eulerian model may only accurately model fluid flows that may be represented with the Euler equations for fluid flows, namely adiabatic and inviscid flows [21], and suffer when computing accurate and dissipation-free advection [20]. As such, future references to Eulerian models in this report will refer to hybrid Eulerian-Lagrangian models whose basis for modeling the fluid stems from a grid-based approach but may implement semi-Lagrangian advection. A summary of this comparison may be seen below in Table 1. Ultimately, we decided to go with a pure Lagrangian fluid simulation, as we felt it was more in-line with the project's primary goal (heterogenous collision detection) and that it would give us more precision and control over the model.

Lagrangian	Eulerian
• Involves simulating individual particles with their own momentums and forces • Computationally intensive, each individual particle needs to consider force contributions each particle surrounding it, but is incredibly precise • Modeling forces can be complex and requires extensive tuning, has huge computational overhead if left unoptimized • Particle collisions can be modeled as a radius around each individual particle, where a particle is "colliding" if it is within the radius and as such, is imposing a non-zero force on another particle*	• Involves using a stationary grid which mimics fluid properties at each grid point, focusing on the flow of fluid volumes through cells in space • Easy to compute, though far less precise than Lagrangian simulations and cannot track individual fluid elements/particles. • Simple models may struggle with viscous flow • Does not actually involve collision detection in a traditional sense, but rather averages fluid vectors in the grid to simulate fluid properties and movement

Table 1. High-level comparison of Lagrangian-based fluid simulations against Eulerian-based approaches. *Forces are approximated to zero at a large enough distance bounded by this radius

Collision Detection & Parallelization Opportunities

A key point to the Lagrangian to Eulerian fluid simulation comparison involves the way both methods treat collision detection. In Lagrangian models, this is represented as a radial kernel gradient called a *smoothing kernel* around a given particle; where a particle is said to collide with another particle if it is within the *core radius* of the smoothing kernel of said particle [22]. If the particle is outside the core radius, the smoothing radius evaluates to zero. In Eulerian models, particle-to-particle collisions do not actually occur. Instead, approximate velocities are averaged over an area of grids to determine the net direction of fluid movement. Solid objects are treated as boundaries where vector lengths towards them are zero (assuming non-moving solids) [18]. Naturally, the way collision detection occurs for both models has huge implications on the opportunities for parallelization.

For Lagrangian models, being within a particle's core radius is analogous to colliding with that particle. This reduces the problem of collision detection to simply calculating the distance between individual particles, or individual points in a space. As a result, for Lagrangian models, it is possible to leverage and apply, with few modifications, a wide array of broad-phase collision detection methods and directly implement existing parallelization techniques for those methods. This includes spatial subdivision and all its variants, which are well-researched and even have existing implementations for particle-based collision detection [19], [23], [24], [25].

Eulerian-based models on the other hand, are simply large vector grids. No "collision detection" exists in the traditional sense between fluid particles, as fluids are just the combination of several vector fields which comprise the fluid's physical properties and momentum at given points in the grid [18], [20]. For solid objects, it is less detection and more a consideration of what exists within each cell and adjacent cells in the stationary grid; e.g. liquid, air, or solid. Contemporary hybrid-Eulerian-Lagrangian methods generally consider *Neumann* boundary conditions for the velocities of solid objects, essentially treating them as immovable objects and preventing any vector flow of liquid into those cells marked as "solid" [18], [20]. Additionally, the models are also reducible into Laplacian matrices, of which solvers are well-understood and parallelization is well-documented [18].

Comparing Lagrangian and Eulerian models, both are intuitive to parallelize and approaches to parallelizing them are well-known, however Eulerian models may not be fully compatible with the capstone project as no real collision detection is used in that family of models, as every cell is always being accounted for regardless. As a result, the safer bet of the two methods is the Lagrangian model, as the Eulerian model may be too tangentially related to the core topic of the capstone project (in relation to collision detection) and may not be explicit enough in that regard to fulfill the needs of the topic.

Numerical Stability

For this section, the *numerical stability* of the fluid model will refer to the fluid simulation's approach to stabilization (i.e. how the model reaches a steady state) as well as the model's behaviour when using large time steps between calculations (as larger time steps enable fewer calculations and therefore give us longer deadlines - more leeway for latency in the case of a real-time simulation). Both of these two concepts are very closely related, and as such will be discussed under a single topic header. In general, model *stability* (in the sense of orderliness and ability to reach a steady state) improves with smaller time steps, at the cost of performance, and vice-versa. Since direct comparisons will be difficult to draw between the two models, analysis will be empirical and focus will be put on conceptual understanding of both models' limitations. Specifically, we will be comparing Lagrangian smoothing kernels with Eulerian successive over-relaxation (SOR), as well as both model's approaches to time steps.

The Lagrangian smoothing kernel is a radial gradient kernel centered on a given particle that generates a vector with variable length when another particle is within its core radius. The vector's length is inversely proportional to the distance of the particle within the radius, and the summation of these vectors for all particles surrounding the center particle is used as the basis for calculating forces such as pressure force and properties such as density at a given region in the simulation [18], [19], [22]. This means that the smoothing kernel serves the dual purpose of preventing particles from bunching up too closely (which would create uneven fluid densities and violate fluid incompressibility), while also ensuring that fluid momentum is somehow lost instead of conserved indefinitely (which prevents liquid simulations from behaving too much like gasses) [19], [22]. A key figure outlining some popular smoothing kernel functions (with scalar distance as input and scalar magnitude as output) can be seen in Figure 4.

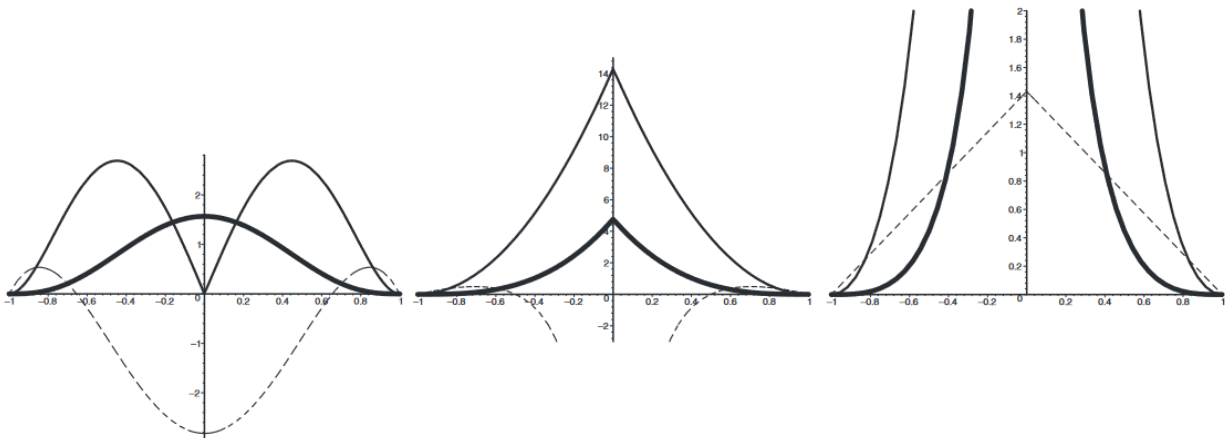


Figure 4. Different types of Lagrangian smoothing kernels from left to right: poly6, spiky, and viscosity; so named after their appearance and use-case. x-scale is scalar distance and y-scale is scalar magnitude. The purpose for the shapes of each kernel is described more in [22].

A popular method for finding the largest-possible time step to use in the Lagrangian model is a heuristic known as the Courant-Friedrichs-Lewy (CDL) condition. The equation is given below by (1) and is adaptive (may also be calculated in real-time repeatedly), but it essentially sets a limit for particle

movement during a given time step to be less than the maximum diameter of a particle for $\lambda=1$ (i.e. one core distance) [19].

$$\Delta t \leq \lambda \frac{h}{\|v^{max}\|} \quad (1)$$

Where h is the core radius, v^{max} is the maximum particle velocity, and λ is some user-set scaling parameter. Typically this can be heuristically chosen (i.e. trial and error) to keep the simulation stable [18], [19], however approaches with an arbitrarily chosen constant time value have been shown to work as well in [22]. The obvious consequence of an excessively large time step is that particles moving quickly may ignore the physics of other particles and hard surfaces, causing them to travel past or directly through other objects.

The Eulerian approach to stabilizing the simulation is typically through the Gauss-Seidel method of successive over-relaxation, SOR, or otherwise a regular conjugate gradient method, with the goal being to quickly solve the linear systems of equations governing the fluid simulation physics and reach a convergent steady-state with each time step [18]. An interesting point in Eulerian simulations is that time step dependencies largely stem from their usage of Lagrangian models for advection, which in turn result in a nearly identical time step requirement as Lagrangian approaches. One approach takes maximizing the time step as minimizing the maximum grid velocity, which is possible in the Eulerian approach by shifting the Eulerian grid to compensate for Lagrangian velocities [21]. Otherwise, time step calculations are largely the same [18], [19], [21].

The general stability of the two models is largely equal, but preference may be given to the Lagrangian method in that adjustments to time steps and stabilizing functions may be easier to notice and debug due to the actual simulation of individual particles, versus the Eulerian approach where grids locations simply embody fluid properties, and actual interactions are hidden in complex formulas evaluated in the background, making small issues difficult to notice. It should be noted that this is double-edged, as it means issues with the former may be easier to notice and therefore problems may be more glaring than in the latter, where the subtlety may help mask them. As a result, stability did not take much precedence in deciding on the model for us to choose, as both models were quite similar in their limitations, and the issue of whether Eulerian models adequately modeled collision detection as well as whether each model fit our actual use-case was more important.

Best Use Cases

The best use case of each model can be determined empirically by analyzing each model's limitations, but there is no need when such analyses have already been done by those smarter and more qualified in-industry.

According to [18], "[Eulerian] simulations are highly accurate, although relatively slow compared to [Lagrangian] solutions. [Lagrangian] simulations are typically much faster, but typically do not look as good as [Eulerian] simulations". The latter remark on appearances, it claims, is because "[Eulerian] simulations often do much better tracking smooth water surfaces, whereas [Lagrangian] approaches often have issues with these smooth surfaces". This somewhat implies that Lagrangian simulations are indeed better for real-time applications, being further reinforced by points from [24]: "Eulerian methods can be made interactive for complex flows [Crane et al. 2007] and can efficiently generate turbulent effects. However, they typically are confined to a finite rectilinear domain and therefore are hard to use in large environments with unconstrained fluid motion. Grid reshaping and adaptive resolution techniques exist for Eulerian methods, but these techniques are slower than regular grid methods due to their use of sparse data structures, and to date have not been demonstrated to run at interactive rates". It is clear that for unconstrained fluid simulations, Eulerian methods are not compatible with real-time applications as they are computationally too expensive.

In contrast to Lagrangian methods: "[Lagrangian] methods like Smoothed Particle Hydrodynamics (SPH) are attractive because they do not suffer from the limitation to be inside a box, however, exactly enforcing incompressibility is important for accurate production of turbulence, and SPH methods have a hard time enforcing incompressibility efficiently. They also can have difficulty allocating computational elements throughout space efficiently. For these reasons, they have not been demonstrated to be effective for calculating single phase flows such as air around a car" [24]. Lagrangian methods are easier to compute, but have key shortcomings in terms of efficiency and for surface tracking and surface rendering at interactive rates [22]. That said, workarounds are described in [19] and [22] which may overcome some of these challenges either through spatial subdivision for the former, or through approximating surface normals and point splatting for the latter.

The overall conclusion is that for real-time, interactive applications, Lagrangian simulations are overwhelmingly dominant due to computational simplicity and the wide array of available optimizations, particularly for collision detection (described in the earlier section). As such, we have decided that our approach will lean into the Lagrangian approach for optimal compatibility with the capstone topic, and ease of modeling and implementation.

Tooling Comparisons

The following sections will be devoted to performing tooling comparisons between various IDE's, including some brief discussion on topics that may not be fully covered by other sections (e.g. language comparisons, version control, etc.).

Version Control

The topic of version control is often immediately answered with “Git”, as version control options can seldom compete with the sheer quantity of features and ease of use of Git. Git is a free, open source distributed version control system that can be easily paired with Github, which is a developer platform (free for noncommercial use) that adds extra features such as access control, bug tracking, software feature requests, task management, and continuous integration [26]. The only meaningful competitor is Perforce - a monolithic version control software tool with web-based repository management, whose main target user is business and enterprise [27]. A comparison table can be found below in Table 2.

Github	Perforce
• Distributed version control model • Free (for non-commercial use) • 24/7 enterprise support, with additional community-based bulletin-board-style support forums • Built on open-source software and version control system, Git • Support primarily for software feature and software debugging features, few project management tools • High familiarity for existing team members (all have used Git and Github before)	• Monolithic server-based version control model • Free (for up to 5 users and 20 workspaces) • 24/7 enterprise support • Fully vertically integrated version control system • Support for integrations with a wide array of project management tools (specifically under the Atlassian company) • Medium familiarity for team members (some have not used Perforce before)

Table 2. Comparison table of the Github and Perforce version control platforms.

It is difficult to justify the use of Perforce, in part due to lack of familiarity with the tool for multiple team members, and in part due to the lack of features we would reasonably use or care about (most are project management-type features which we do not really need). As such, even though Perforce is a relatively competitive version control system in terms of feature quantity, the team naturally favours Git and Github due to familiarity and broader feature set for our use cases.

There were other options, such as Unity Version Control (previously Plastic SCM), however such options often lacked features and were too vertically integrated with a specific platform, which would lock us into a specific development platform (e.g. Unity engine), which is something we were not comfortable with doing too early in the project.

Independent Development Environment (IDE)

There are several decent options for IDE. This choice is largely left for group members to choose personally, as IDE choice has little effect on the project as a whole (so long at least one IDE used has some form of performance profiling tools). A comparison table with some popular options (for game development) can be found below (Table 3).

Visual Studio Code	Visual Studio	JetBrains Rider
• Standalone multi-platform code editor • Supports user-made extensions, enabling vast customization and support for virtually any programming language ◦ No reliable (customer) support for user plugins • Free • Simple and easy to use • Support for Github Copilot • Built-in code refactoring • Native performance profiling tools (less extensive than Visual Studio)	• All-in-one feature-laden IDE • 24/7 enterprise support • Native compilers • Built-in graphic design tools • Native support for: ◦ C#, F#, .NET, NuGet ◦ Unity • Built-in debuggers for each natively supported platform • Free (for noncommercial use) • Support for Github Copilot • Built-in code refactoring • Tight Git/GitHub integrations • Native testing tools for unit testing • Support for collaborative coding (multiple writers) • Native performance profiling tools	• All-in-one feature-laden IDE • 24/7 enterprise support • Fully vertically integrated IDE environment • Native support for: ◦ C#, F#, .NET, NuGet ◦ Unreal Engine, Unity, Custom C++, Godot • Built-in debuggers for each natively-supported platform • Expensive (~120 US / yr) • In-house AI tooling for quality of life and code suggestions • Built-in “code analysis” tools for live code-linting across all natively supported platforms • Native decompiler support for viewing third-party libraries • Built-in code refactoring • Support for a broad set of version control options (Git, Subversion, Mercurial, Perforce, and TFS)

Table 3. A brief IDE comparison chart [28, 29, 30].

It is recommended that group members use Visual Studio, as it has in-built support for developing Unity applications, is free, and has an extensive performance profiler suitable for this project's needs.

Software Language

The number of languages worth considering for such a project are few and far apart. The primary two contenders are C#, and C++. This is primarily because of library support (Unity-based HLSL for the former and OpenCL for the latter), however library discussions have already been covered in previous sections. This section will instead loosely compare the two languages. A comparison table can be found below in Table 4:

C#	C++
1. C# is generally easier to write and maintain compared to C++. Its memory management via garbage collection reduces common errors like memory leaks and dangling pointers, making it more beginner-friendly. 2. While C# can be optimized, it still typically lags behind C++ in raw performance, especially in compute-intensive applications like fluid simulations. However, with GPU support via frameworks like ComputeSharp (which uses DirectCompute, DirectX 12, or Vulkan) or libraries like Slik.NET, you can leverage the GPU for certain tasks. 3. C# has good tooling and integration with the .NET ecosystem and Unity. It's ideal for applications where rapid development and higher-level abstractions are more important than squeezing out every bit of performance.	1. C++ is much closer to the metal than C#. It gives you full control over memory and CPU/GPU optimizations. For fluid simulations, where computational efficiency is key, C++ can deliver higher performance due to its ability to interact directly with hardware and avoid overhead from runtime environments like .NET. 2. C++ is the standard for high-performance GPU programming. You can directly leverage CUDA, OpenCL, and Vulkan for GPU computation 3. C++ is more complex to write, debug, and maintain compared to C#. Memory management, pointers, and concurrency can introduce bugs and require careful handling.
Summary: C# is suitable for rapid prototyping, easier debugging.	Summary: Ideal when performance is the primary concern

Table 4. Comparison of the C# and C++ in the context of graphical programming [31]

We will use C#, because all demanding parallelizable compute tasks will be parallelized to the GPU, and C# will mainly be used to interface with the Unity engine API to set up the environment and its properties, and to create readable, debuggable input management. And we want a solution that can be easily ported to other games.

3. **Difficulties Encountered in Reporting Period** (Provide detailed information on the difficulties and issues that you encountered during the reporting period and how you plan to address this in the following periods)

Group:

Student A:

Some challenges during this period included reading through dense research papers on fluid simulation and collision detection, as well as reaching a consensus on certain implementation specifics, such as the choice of game engine, after much debate.

Student B:

I had to dig deeply into Unity's documentation to understand how its pricing model differed from Unreal, since the subscription tier eligibility is hidden. It was also difficult to calculate where our cost regions differed between Unity and Unreal as well as the advantages of each.

Finally, I had to read much of the documentation for Unity and Unreal to compare them. Reading a lot is difficult.

Student C:

Since this report focused on research to determine the best graphics API to use for our project and determine whether a game engine should be used, there were no significant difficulties encountered. The main difficulties for this period were researching collision detection and different graphics APIs to evaluate which would be best suited for the project.

Student D:

Due to the sheer quantity of research material regarding fluid simulations, alongside the difficulty in understanding the contents of several papers (primarily due to assumptions that readers have some prior knowledge of the field), it was difficult to properly comprehend the contents of each paper on the first read. This is not a significant issue for later periods, as learning is an inherently incremental process, and thus the problem can be said to “solve itself” over time; however possible solutions that have worked thus far were to read secondary and tertiary sources on the more complex topics, and slowly understand every concept detailed in each paper incrementally, instead of trying to understand the same paper through repeated readings.

4. **Tasks to Be Completed in the Next Reporting Period** (Outline the tasks to be completed in the next reporting period. Please note this should match with your milestones submitted to your FLC in Week3, however in consultation with (and approval of) your FLC, you can modify this to accommodate incomplete tasks from previous period. Here you should also identify the Project Manager for the next period)

Group:

Student A:

1. Decompose Research Papers into Workable Tasks
2. Break down complex concepts into manageable components.
3. Identify the core ideas or algorithms within each research paper and break them into smaller, executable tasks (e.g., implementing specific optimization methods).
4. Smoothed Particles
 - a. Task: Refine the smoothing techniques used for particles in simulation.
 - b. Test different smoothing kernels to reduce artifacts in particle interactions.
5. Optimize kernel functions for GPU processing to enhance real-time performance.

Student B:

1. Assign task types.
2. Separate tasks into “Research,” “Prototype Development,” and “Optimization” categories.
3. Research Task: Study spatial subdivision algorithms and their impact on reducing unnecessary collision calculations.
4. Development Task: Implement basic spatial subdivision in Unity.
5. Optimization Task: Profile and test the implementation on both CPU and GPU.

Student C:

1. Timeline and priority.
 - a. Set priority levels for each task and assign deadlines to make progress systematic.
2. Link tasks to the overall project.
 - a. Ensure each task directly contributes to the end goal of optimizing collision detection for fluid simulations.
3. The Interpolation Equation
 - a. Fine-tune interpolation methods to improve accuracy.
 - b. Explore different interpolation techniques for particle properties like velocity and pressure.
 - c. Implement parallelized interpolation on the GPU.

Student D:

1. Reevaluation and Optimization of Key Fluid Simulation Components
 - a. Reevaluate the gravity force implementation, ensuring realistic interaction with particle-based collisions.
 - b. Investigate gravitational force formulas used in existing simulations.
 - c. Test GPU acceleration for gravity-based interactions between particles.
2. Gradient Calculations
 - a. Identify potential optimizations for gradient-based interactions in fluid particles.
 - b. Profile GPU-based gradient calculation for improved performance.

References

- [1] “Video Game Market Size, Share & Trends Analysis Report By Device (Console, Mobile, Computer), nBy Type (Online, Offline), By Region (Asia Pacific, North America, Europe), And Segment Forecasts, 2023 - 2030,” Grand View Research, San Francisco, CA, USA, Rep. GVR-4-68038-527-4, 2024. Accessed: Oct. 2, 2024. [Online]. Available: <https://www.grandviewresearch.com/industry-analysis/video-game-market>
- [2] “Graphic Processing Unit Market Size Expected to Reach USD 1,414.39 Bn by 2034,” Precedence Research, Ottawa, ON, Canada, Rep. 2939912, 2024. Accessed: Oct. 2, 2024. [Online]. Available: <https://www.globenewswire.com/en/news-release/2024/09/03/2939912/0/en/Graphic-Processing-Unit-GPU-Market-Size-Expected-to-Rreach-USD-1-414-39-Bn-by-2034.html>
- [3] Unity Technologies, San Francisco, CA, USA. *Unity Manual 2022.3 LTS*. (2024). Accessed: Oct. 2, 2024. [Online]. Available: <https://docs.unity3d.com/Manual/>
- [4] A. Marian, “Unity runtime on Arm-based Windows devices.” Unity Technologies. Accessed: Oct 2, 2024. [Online]. Available: <https://unity.com/blog/engine-platform/unity-runtime-on-arm-based-windows-devices>
- [5] K. Karimi, N. G. Dickson, and F. Hamze, “A Performance Comparison of CUDA and OpenCL,” D-Wave Systems Inc., Burnaby, BC, Canada, arXiv:1005.2581. [Online]. Available: <https://arxiv.org/pdf/1005.2581>
- [6] J. Fang, A. L. Varbanescu, and H. J. Sips, “A comprehensive performance comparison of CUDA and OpenCL,” *Int. Conf. on Parallel Processing*. Taipei, Taiwan, Sept. 13-16, 2011, doi: 10.1109/ICPP.2011.45. [Online]. Available: https://www.researchgate.net/publication/221084751_A_comprehensive_performance_comparison_of_CUDA_and_OpenCL
- [7] stevewhims, alexbuckgit, DCtheGeek, mijacobs, and msatranjr. “High-level shader language (HLSL).” High-level shader language (HLSL) - win32 apps. Win32 apps | Microsoft Learn. Accessed: Oct. 2, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/direct3dhls/dx-graphics-hlsl>
- [8] Wikipedia contributors, “CUDA.” Wikipedia. Accessed Oct. 2, 2024. [Online]. Available: <https://en.wikipedia.org/wiki/CUDA>
- [9] M. R. Varela, G. Sitaraman, and B. Robey, “Application portability with HIP.” GPUOpen. Accessed Oct. 2, 2024. [Online]. Available: <https://gpuopen.com/learn/amd-lab-notes/amd-lab-notes-hipify-readme/>
- [10] Wikipedia contributors, “High-level Shader Language.” Wikipedia. Accessed Oct. 2, 2024. [Online]. Available: https://en.wikipedia.org/wiki/High-Level_Shader_Language
- [11] Intel, Santa Clara, CA, USA. *Intel FPGA SDK for OpenCL*. (2022). Accessed: Oct. 2, 2024. [Online]. Available:

- <https://www.intel.com/content/www/us/en/support/programmable/support-resources/design-guidance/openccl-software-support.html>
- [12] "Graphics Cards, Laptops, and Desktops." Nvidia Store. (2024). Accessed Oct. 2, 2024. [Online]. Available: <https://store.nvidia.com/en-us/consumer/?page=1&limit=12&locale=en-us>
- [13] A. Galvan, "A review of shader languages." Alain.xyz. (May 21, 2024). [Online]. Available: <https://alain.xyz/blog/a-review-of-shader-languages>
- [14] N. Trevett, A. Richards, M. Butler, J. McVeigh, A. Bhat, B. Calidas, V. Hindriksen, and W. Dai, "Open Standard for Parallel Programming of Heterogeneous Systems." Khronos Group Inc. (Jul. 21, 2023). Accessed: Oct. 2., 2024. [Online]. Available: <https://www.khronos.org/openccl/>
- [15] Wikipedia contributors, "Game Engine." Wikipedia. Accessed Oct. 2, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Game_engine
- [16] J. Halpern, "The what and why of Game Engines," medium.com. (Dec. 11, 2018). [Online]. Available: <https://medium.com/@jaredehalpern/the-what-and-why-of-game-engines-f2b89a46d01f>
- [17] SteamDB, 2024. "Game Releases 2024," distributed by SteamDB. Accessed Oct. 2, 2024. [Online]. Available: <https://steamdb.info/instantsearch/?refinementList%5BappType%5D%5B0%5D=Game&refinementList%5BreleaseYear%5D%5B0%5D=2024>
- [18] C. Braley, A. Sandu. (2009). Fluid Simulation for Computer Graphics: A Tutorial in Grid Based and Particle Based Methods [Online]. Available: https://cg.informatik.uni-freiburg.de/intern/seminar/gridFluids_fluid-EulerParticle.pdf
- [19] D. Koschier, J. Bender, B. Solenthaler, and M. Teschner. (2019). Smoothed Particle Hydrodynamics Techniques for the Physics Based Simulation of Fluids and Solids [Online]. Available: https://sph-tutorial.physics-simulation.org/pdf/SPH_Tutorial.pdf
- [20] F. Ye, J. Litven, D. I. W. Levin, and D. K. Pai, "Eulerian-on-Lagrangian Simulation," *ACM Transactions on Graphics*, vol. 32, no. 3, pp 1-9, Jul. 4, 2013, doi: 10.1145/2487228.248723. [Online]. Available: https://www.cs.ubc.ca/~yefan/EoL_preprint.pdf
- [21] Wikipedia contributors, "Euler equations (fluid dynamics)." Wikipedia. Accessed Oct. 2, 2024. [Online]. Available: [https://en.wikipedia.org/wiki/Euler_equations_\(fluid_dynamics\)](https://en.wikipedia.org/wiki/Euler_equations_(fluid_dynamics))
- [22] J. M. Cohen, S. Tariq, and S. Green, "Interactive Fluid-Particle Simulation using Translating Eulerian Grids," presented at the 2010 ACM SIGGRAPH symposium on Interactive 3D Graphics and Games, doi.org/10.1145/1730804.1730807. [Online]. Available: https://research.nvidia.com/sites/default/files/pubs/2010-02_Interactive-Fluid-Particle-Simulation/paper.pdf
- [23] H. Nguyen and S. Le Grand, "Broad-Phase Collision Detection with CUDA," in *GPU Gems 3*, 1st ed. Boston, MA, USA: Addison-Wesley Professional, 2007, ch. 32. Accessed Oct. 1, 2024. [Online]. Available:

<https://developer.nvidia.com/gpugems/gpugems3/part-v-physics-simulation/chapter-32-broad-phase-collision-detection-cuda>

- [24] M. Müller, D. Charypar, and M. Gross, "Particle-Based Fluid Simulation for Interactive Applications," presented at the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation, July 26-27, 2003. [Online]. Available:
<https://matthias-research.github.io/pages/publications/sca03.pdf>
- [25] M. Tayyub and G. N. Khan, "Heterogeneous Design and Efficient CPU-GPU Implementation of Collision Detection," IADIS International Journal on Computer Science and Information Systems, Vol. 14, No. 2, pp. 25-40, July-Dec. 2019. [Online]. Available:
<https://www.iadisportal.org/ijcsis/papers/2019140202.pdf>
- [26] "GitHub features," GitHub. Accessed Oct. 2, 2024. [Online]. Available:
<https://github.com/features>
- [27] "Perforce helix core," Perforce. Accessed Oct. 2, 2024. [Online]. Available:
<https://www.perforce.com/products/helix-core>
- [28] "Visual Studio 2022 IDE," Microsoft. Accessed Oct. 2, 2024. [Online]. Available:
<https://visualstudio.microsoft.com/vs/>
- [29] "Visual Studio Code," Microsoft. Accessed Oct. 2, 2024. [Online]. Available:
<https://code.visualstudio.com/>
- [30] "Features - Rider," JetBrains. Accessed Oct. 2, 2024. [Online]. Available:
<https://www.jetbrains.com/rider/features/>
- [31] "C# Vs. C++: The Difference Between C++ and C#," Simplilearn. Accessed Oct. 2, 2024. [Online]. Available: <https://www.simplilearn.com/tutorials/cpp-tutorial/c-sharp-vs-cpp>